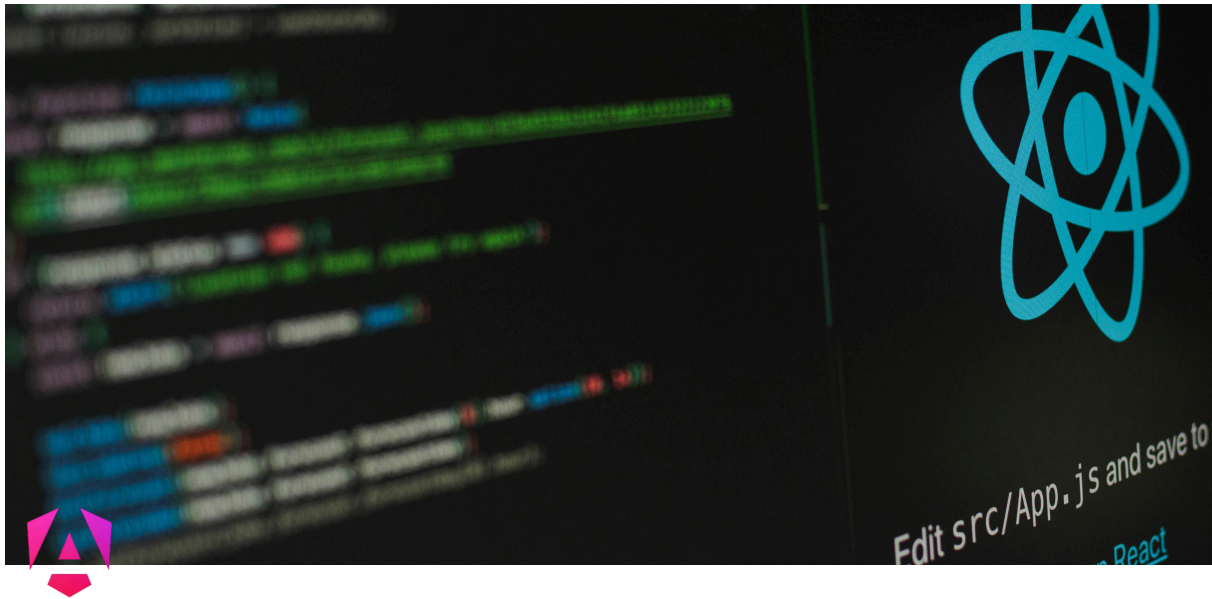




Angular

🕒 Last edited time	April 17, 2026 10:10 AM
⚙️ Status	Em andamento

<https://dontpad.com/aulinhaAngular>

CLI

1. Compreender ArquiteturaAngular e seus comandos

O Angular é um framework baseado em componentes, serviços e injeção de dependências. Uma aplicação bem arquitetada (como a do seu repositório) geralmente é dividida em diretórios lógicos:

- **Domain/Types:** Onde ficam as interfaces (contratos de dados). Ex: `pixel.model.ts`.
- **Features:** Módulos isolados da aplicação (ex: `auth`, `board`, `profile`).
- **Shared:** Componentes reutilizáveis em qualquer lugar (ex: um botão customizado, um pipe de formatação).
- **Services:** Classes que lidam com lógica de negócios e chamadas externas (API, WebSockets).
- **Guards:** "Seguranças" das rotas (ex: só entra no rPlace se estiver logado).

Principais Comandos CLI:

- Criar projeto: `ng new rplace-app`
- Criar componente (standalone): `ng g c features/board/components/canvas --standalone`
- Criar serviço: `ng g s core/services/websocket`
- Criar guard: `ng g g core/guards/auth`

Checklist

1. Setup e Arquitetura (Competências 1, 4 e 5)

- ☐ **Criação do Projeto:** Garantiu que o projeto é Standalone (Angular 14+ ou 17+).
- ☐ **Estrutura de Pastas:** Criou as pastas `features` (lógica de tela), `shared` (reutilizáveis), `core` (serviços e guards) e `domain` (interfaces/types).
- ☐ **Rotas (SPA):** Configurou o arquivo `app.routes.ts` com as rotas para 'Login' e 'Board'.
- ☐ **Standalone Components:** Verificou se todos os componentes têm a propriedade `standalone: true` e se os imports necessários (CommonModule, FormsModule) estão dentro do array `imports: []` de cada componente.

2. Fundamentos e Comunicação entre Componentes (Competência 2)

- ☐ **Interfaces (Types):** Criou a interface no diretório `domain`.
- ☐ **@Input():** No componente de Pixel (filho), criou os inputs para receber dados
- ☐ **@Output() / EventEmitter:** Criou o evento para avisar o componente pai.
- ☐ **Property & Event Binding:**
 - No HTML do filho: usou `[style.background-color]` para a cor.
 - No HTML do filho: usou `(click)` para disparar a função de emissão.
 - No HTML do pai: usou `(nomeDoEvento)="minhaFuncao($event)"` para capturar o clique.

3. Formulários e Estado Local (Competências 3 e 6)

- ☐ **Reactive Forms:** No componente de Login, injetou o `FormBuilder` e criou o `FormGroup` com validações (`Validators.required`).
- ☐ **Binding de Form:** Conectou o `[formGroup]` no HTML e os `formControlName` nos inputs.
- ☐ **Signals:** Criou um serviço (ou variável no componente) usando `signal<string>('#000000')` para gerenciamento.
- ☐ **Leitura de Signals:** Garantiu que no HTML a leitura é feita com parênteses: `{{ meuSignal() }}`.

4. Comunicação Assíncrona e HTTP (Competências 7 e 8)

- ☐ **HttpClient:** Injetou o `HttpClient` no serviço principal.
- ☐ **Observables (GET):** Criou uma função que retorna um `Observable` da API (ex: `getBoard()`).
- ☐ **Subscription:** No `ngOnInit` do componente pai, deu um `.subscribe()` no método do serviço para preencher o quadro inicial.
- ☐ **Pipes (RxJS):** Usou operadores como `map` ou `tap` para tratar os dados antes de chegarem ao componente.

5. Tempo Real e Mensageria (Competências 9 e 10)

- ☐ **WebSocket Connection:** Usou o `websocket` do `rxjs/websocket` para abrir a conexão no serviço.
- ☐ **Escuta Ativa:** Criou um Observable (ou Subject) que fica "ouvindo" as mensagens que chegam do servidor sobre novos pixels pintados.
- ☐ **Atualização Reativa:** Ao receber um dado via WebSocket, utilizou o método `.update()` do seu Signal de pixels para atualizar a tela sem recarregar nada.

6. Segurança e Controle (Extras mencionados)

- ☐ **Auth Guard:** Criou um arquivo `.guard.ts` que implementa `CanActivateFn` para proteger a rota do Board.
- ☐ **Service Logic:** Garantiu que a lógica de "pintar" (fazer o POST ou mandar pro Socket) está dentro do serviço, e não dentro do componente (princípio de responsabilidade única).

Componentes

2. Elaboração de Componentes e Componentes Standalone

- **que é um Standalone Component?** É um componente que não precisa ser declarado em um arquivo `module.ts` (`@NgModule`). Ele importa suas próprias dependências, tornando o código muito mais limpo.

Binding: É a comunicação entre a sua classe TypeScript (`.ts`) e o seu HTML (`.html`).

- **Interpolation** `{{ valor }}` :: Mostra uma variável do TS no HTML.
- **Property Binding** `[propriedade]="valor"` :: Envia um dado do TS para uma propriedade do HTML.
- **Event Binding** `(evento)="funcao()"` Escuta uma ação do HTML (como um clique) e chama uma função no TS.
- **@Input:** Permite que um componente "Pai" passe dados para um componente "Filho".
- **@Output:** Permite que o "Filho" avise o "Pai" que algo aconteceu (usando o `EventEmitter`).

`pixel.component.ts` (Filho)

```
import { Component, Input, Output, EventEmitter } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-pixel',
  standalone: true, // Moderno: não precisa de NgModule!
  imports: [CommonModule],
  templateUrl: './pixel.component.html',
  styleUrls: ['./pixel.component.css']
})
export class PixelComponent {
  // @Input() permite que quem usar <app-pixel> injete uma cor.
  // Ex: <app-pixel [color]="'#FF0000'"></app-pixel>
```



```

    @Input() color: string = '#FFFFFF'; // Branco por padrão
    @Input() x!: number; // Coordenada X (a exclamação diz qu
e vamos receber depois)
    @Input() y!: number; // Coordenada Y

    // @Output() cria um evento customizado. O pai vai escuta
r isso.
    // Ex: <app-pixel (pixelClicked)="pintarPixel($event)"></
app-pixel>
    @Output() pixelClicked = new EventEmitter<{x: number, y:
number}>();

    // Função chamada pelo HTML quando o usuário clica neste
pixel
    onClick() {
        // Avisa o Pai emitindo as coordenadas deste pixel
        this.pixelClicked.emit({ x: this.x, y: this.y });
    }
}

```

pixel.component.html (Filho)

```

<div
  class="pixel-box"
  [style.background-color]="color"
  (click)="onClick()">
</div>

```

Forms

Competência 3: Formulários Reativos (Reactive Forms)

A lógica inteira do formulário (validações, campos) fica no TypeScript, e o HTML apenas "reflete" essa estrutura.

Cenário: O usuário precisa digitar um "Username" para entrar no rPlace.

login.component.ts

```

import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators, ReactiveFormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule], // Necessário para Reactive Forms
  templateUrl: './login.component.html'
})
export class LoginComponent {
  loginForm: FormGroup; // Guarda todo o estado do formulário

  // Injetamos o FormBuilder (para facilitar a criação) e o Router (para mudar de página)
  constructor(private fb: FormBuilder, private router: Router) {
    // Criamos o formulário
    this.loginForm = this.fb.group({
      // Campo 'username', começa vazio, e é Obrigatório (Required)
      username: ['', [Validators.required, Validators.minLength(3)]],
      // Campo 'color', começa com preto
      defaultColor: ['#000000']
    });

    onSubmit() {
      // Se o form for válido, pegamos os dados e vamos para o jogo
      if (this.loginForm.valid) {
        console.log('Dados do form:', this.loginForm.value);
        // Aqui chamaríamos um serviço de Autenticação (auth) e depois:
      }
    }
  }
}

```

```

        this.router.navigate(['/board']);
    }
}
}

```

login.component.html

```

<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">

  <label>Nome de usuário:</label>
  <input type="text" formControlName="username">

  @if (loginForm.get('username')?.hasError('required') && loginForm.get('username')?.touched) {
    <span class="error">O nome é obrigatório!</span>
  }

  <label>Cor padrão:</label>
  <input type="color" formControlName="defaultColor">

  <button type="submit" [disabled]="loginForm.invalid">Entrar no rPlace</button>

</form>

```

Quando usar: Formulários de login, cadastro de usuário, ou painéis de configuração onde você precisa de validação complexa e quer reagir a mudanças em tempo real no que o usuário digita.

Competência 4 e 5: SPA e Componentes Standalone

SPA (Single Page Application): O site nunca recarrega a página inteira. O Angular apenas destrói componentes antigos e desenha os novos na tela. É muito mais rápido.

- **Standalone Components:** O padrão atual do Angular. Antigamente, você precisava registrar tudo em um arquivo chamado `app.module.ts`. Agora, os componentes são independentes. Eles importam apenas o que vão usar (como fizemos no array `imports: []` acima).

Cenário: Configurando as rotas do rPlace sem módulos.

app.routes.ts

```
import { Routes } from '@angular/router';
import { LoginComponent } from '../features/login/login.component';
import { BoardComponent } from '../features/board/board.component';
import { authGuard } from '../core/guards/auth.guard'; // Exemplo de Guard

export const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', component: LoginComponent },
  {
    path: 'board',
    component: BoardComponent,
    canActivate: [authGuard] // O Angular roda esse Guard antes de entrar. Se retornar false, bloqueia.
  }
];
```

Competência 6: Signals (Otimização de Renderização)

O que é? Signals são a nova forma do Angular gerenciar "Estado" (variáveis que mudam e precisam atualizar a tela). Quando um Signal muda, o Angular atualiza **exatamente e apenas** a parte do HTML que usa aquele Signal. É brutalmente mais rápido e performático que variáveis comuns.

Cenário: O usuário escolhe uma cor na paleta de cores. Queremos armazenar a "cor atual" que ele está segurando na mão para pintar.

color-state.service.ts (Serviço para guardar o estado)

```
import { Injectable, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class ColorStateService {
  // Criamos um Signal. Valor inicial é preto.
  // Ele é "writable", ou seja, podemos alterar o valor dele.
}
```

```

public selectedColor = signal<string>('#000000');

// Função para mudar a cor
changeColor(newColor: string) {
  this.selectedColor.set(newColor); // Atualiza o signal
}
}

```

`board.component.ts` (O Pai - usando o signal)

```

import { Component, inject } from '@angular/core';
import { ColorStateService } from '../core/services/color-state.service';

@Component({
  selector: 'app-board',
  standalone: true,
  template: `
    <h2>Cor selecionada atualmente: <span [style.color]="colorService.selectedColor()">{{ colorService.selectedColor() }}</span></h2>

    <button (click)="mudarPraVermelho()">Pegar Vermelho</button>
  `
})
export class BoardComponent {
  // inject() é a forma moderna de injetar serviços no Angular Standalone
  colorService = inject(ColorStateService);

  mudarPraVermelho() {
    this.colorService.changeColor('#FF0000');
  }
}

```

Quando usar: Para qualquer variável cujo valor vai mudar na tela ao longo do tempo de forma síncrona (como itens selecionados, contadores, configurações

de UI).

Competências 7, 8, 9 e 10: Observables, Requisições HTTP, WebSockets e Mensageria

- **Http Client:** Usado para pegar o estado inicial do quadro (o banco de dados retorna uma foto de como o quadro está agora). Requisição única (bate no servidor e a conexão morre).
- **WebSockets / Mensageria:** Usado para comunicação **em tempo real**. O servidor e o front-end ficam com um "tubo" aberto. Se Joãozinho lá no Japão pintar um pixel, o servidor avisa o nosso frontend na mesma hora, sem a gente precisar dar F5.
- **Observables:** Representam um fluxo de dados no tempo. Perfeitos para lidar com respostas HTTP e mensagens do WebSocket.

O `.pipe()` é um método do RxJS disponível em todos os **Observables**. Ele funciona como um **mecanismo de montagem**. Imagine uma linha de produção em uma fábrica: o `pipe` é a esteira rolante.

1. **A Esteira (Pipe):** Quando você chama `.pipe()`, você está dizendo ao Angular: "Prepare este fluxo de dados para receber instruções de processamento".
2. **Os Operadores (Filtros):** Se você coloca algo dentro do pipe (como `map`, `filter`, `tap`), você está colocando máquinas nessa esteira para transformar o produto.

```
import { Injectable } from '@angular/core';
import { Api } from './api';
import { Observable, Subject } from 'rxjs';
import { IPixel } from '../features/main-page/components/pixel/IPixel';

@Injectable({
  providedIn: 'root',
})
export class PixelApi extends Api {

  // Competência 8: Requisição GET (Busca todos os pixels)
  public GetAll = (): Observable<IPixel[]> => {
```

```

        return this.client.get<IPixel[]>(`${this.URL}/pixel`).pipe();
    }

    // Competência 8: Requisição POST (Atualiza/Cria um pixel)
    public UpdatePixel = (data: IPixel): Observable<void> => {
        return this.client.post<void>(`${this.URL}/pixel`, data).pipe();
    }
}

```

Passo 1: A Interface (Domain)

pixel.model.ts

```

export interface PixelDTO {
    x: number;
    y: number;
    color: string;
}

```

Passo 2: O Serviço de Comunicação

rplace.service.ts

```

import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, Subject } from 'rxjs';
import { WebSocket, WebSocketSubject } from 'rxjs/webSocket';
import { PixelDTO } from '../../domain/pixel.model';

@Injectable({ providedIn: 'root' })
export class RPlaceService {
    private http = inject(HttpClient);

    // O WebSocketSubject é um Observable especial para WebSockets
}

```



```

private socket$: WebSocketSubject<any>;

// Subject é um Observable que podemos emitir valores man
ualmente.
// Vamos usar ele para avisar os componentes quando chega
r um pixel novo via socket.
private pixelUpdates = new Subject<PixelDTO>();
public pixelUpdates$ = this.pixelUpdates.asObservable();
// Componentes se inscrevem aqui

// HTTP (Competência 8) - Pega o quadro inteiro uma única
vez ao carregar a página
getInitialBoard(): Observable<PixelDTO[]> {
    return this.http.get<PixelDTO[]>('https://api.seusite.c
om/board');
}

// WEBSOCKET (Competências 9 e 10) - Conecta no servidor
em tempo real
connectSocket() {
    // Estabelece a conexão
    this.socket$ = websocket('wss://api.seusite.com/ws-rpla
ce');

    // Fica escutando as mensagens do servidor
    this.socket$.subscribe({
        next: (message) => {
            // Quando o servidor avisa "Alguém pintou um pixe
l!", nós recebemos aqui
            if (message.type === 'PIXEL_UPDATE') {
                // Repassamos a mensagem para dentro do nosso app
                this.pixelUpdates.next(message.data);
            }
        },
        error: (err) => console.error('Erro no WebSocket', er
r)
    });
}

```

```

// Envia um comando pelo WebSocket para o servidor
drawPixel(pixel: PixelDTO) {
  this.socket$.next({
    type: 'DRAW_PIXEL',
    data: pixel
  });
}
}

```

Passo 3: Juntando tudo no Quadro Principal (Feature)

Aqui usamos o `@Input`, `@Output`, Signals, Observables e Services tudo junto!

`board.component.ts`

```

import { Component, OnInit, inject, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { PixelComponent } from '../pixel/pixel.component';
import { RPlaceService } from '../../core/services/rplace.service';
import { ColorStateService } from '../../core/services/color-state.service';
import { PixelDTO } from '../../domain/pixel.model';

@Component({
  selector: 'app-board',
  standalone: true,
  imports: [CommonModule, PixelComponent],
  templateUrl: './board.component.html',
  styleUrls: ['./board.component.css']
})
export class BoardComponent implements OnInit {
  rPlaceService = inject(RPlaceService);
  colorService = inject(ColorStateService); // Pega a cor que vimos na competência 6

  // Usamos um Signal para guardar a matriz de pixels
  boardPixels = signal<PixelDTO[]>([]);

```

```

ngOnInit() {
  // 1. Conecta no WebSocket
  this.rPlaceService.connectSocket();

  // 2. Busca o tabuleiro inicial via HTTP e atualiza o Signal
  this.rPlaceService.getInitialBoard().subscribe((pixelsDoServidor) => {
    this.boardPixels.set(pixelsDoServidor);
  });

  // 3. Fica escutando atualizações em tempo real pelo WebSocket
  this.rPlaceService.pixelUpdates$.subscribe((novoPixel) => {
    // Atualiza a lista de pixels local com o novo pixel que veio da rede
    this.boardPixels.update(pixelsAtuais => {
      // Encontra o pixel que foi alterado e substitui a cor dele
      return pixelsAtuais.map(p =>
        (p.x === novoPixel.x && p.y === novoPixel.y) ? novoPixel : p
      );
    });
  });

  // Função chamada pelo (pixelClicked) do nosso app-pixel
  onUserClickedPixel(coordenadas: {x: number, y: number}) {
    // Pega a cor atual selecionada no Signal do serviço de cor
    const corSelecionada = this.colorService.selectedColor();

    const pixelParaPintar: PixelDTO = {
      x: coordenadas.x,

```

```

        y: coordenadas.y,
        color: corSelecionada
    };

    // Manda para o servidor via WebSocket (Mensageria)
    this.rPlaceService.drawPixel(pixelParaPintar);
  }
}

```

board.component.html

```

<div class="board-container">

  @for (pixel of boardPixels(); track pixel.x + '-' + pixel.y) {

    <app-pixel
      [color]="pixel.color"
      [x]="pixel.x"
      [y]="pixel.y"
      (pixelClicked)="onUserClickedPixel($event)">
    </app-pixel>

  }

</div>

```

Use **Signals** para guardar estado (como a cor selecionada, ou a matriz de pixels). Use **Observables** para eventos assíncronos que acontecem no tempo (como cliques constantes, respostas de API HTTP ou mensagens chegando de WebSockets).

Rotas

```

import { Routes } from '@angular/router';
import { MainPage } from '../features/main-page/main-page';
import { LoginPage } from '../features/login-page/login-page';

```

```

import { authGuard } from './domain/auth-guard';
import { RoomPage } from './features/room-page/room-page';
import { EspecificRoomPage } from './features/room-page/especific-room-page/especific-room-page';

export const routes: Routes = [
  //! problema era que o guard precisava ser validado por ultimo, pois a ordem da lista importa ja que o match foi rejeitado e as rotas nao foram vistas antes(no caso de login)
  {path: "login", component: LoginPage, canMatch:[authGuard]},
  {path: "", component: MainPage, canMatch:[authGuard]},
  {path: "room", component: RoomPage, canMatch:[authGuard]},
  children: [
    {path: ":id", component: EspecificRoomPage}
  ]},
];

```

Inputs e Outputs

```

import { Component, EventEmitter, Input, Output } from '@angular/core';
import { IPixel } from './IPixel';

@Component({
  selector: 'app-pixel',
  imports: [],
  templateUrl: './pixel.html',
  styleUrls: ['./pixel.css'],
})
export class Pixel {
  @Input()
  data!: IPixel;

  @Output()
  onChange: EventEmitter<IPixel> = new EventEmitter();
}

```

```

    change(event: string){
      this.data.Color = event;
      this.onChange.emit(this.data);
    }
  }
}

```

```

// html pixel (filho)
<input type="color" class="pixel" [value]="data.Color" (change)="change($event.target.value)">

```

```

// output pai (main page)
<header>
  <h1>Pixels</h1>
  <button (click)="logout()">Logout</button>
</header>

<main class="pixels-zone">

  @for (row of pixels; track $index) {
    <div class="row">
      @for (pixel of row; track $index) {
        <app-pixel [data]="pixel" (onChange)="updateData($event)" />
      }
    </div>
  }
</main>

```

Property e EventEmitter e FormModule

```

import { Component, OnInit } from '@angular/core';
import { AuthApi } from '../../domain/auth.api'; // Serviço
que lida com as requisições de login/cadastro
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { LoginDto } from '../../domain/UserInterfaces'; //

```

Interface que define a estrutura dos dados de login

```
import { Router } from '@angular/router';

@Component({
  selector: 'app-login-page',
  standalone: true, // Importante para a Competência 5
  imports: [ReactiveFormsModule], // Necessário para usar
[formGroup] no HTML (Competência 3)
  templateUrl: './login-page.html',
  styleUrls: ['./login-page.css'],
})
export class LoginPage {
  // Injeção de dependência: o Angular fornece as instâncias
da API e do Roteador automaticamente
  constructor(
    private api : AuthApi,
    private router: Router
  ){ }

  // Variável de controle: define se o usuário está na tela
de "Login" ou "Cadastro" (Subscribe)
  protected isSubscribe: boolean = false;

  // --- COMPETÊNCIA 3: FORMULÁRIOS REATIVOS ---
  // Criamos o grupo do formulário no TypeScript para ter c
ontrol total sobre as validações
  loginForm : FormGroup = new FormGroup({
    // Cada FormControl representa um campo do formulário
    // O segundo parâmetro é um array de validadores (campo
obrigatório)
    username: new FormControl('', [Validators.required]),
    password: new FormControl('', [Validators.required])
  })

  // Getters: Facilitam o acesso aos campos no HTML para mo
strar mensagens de erro
  get Username() {
    return this.loginForm.get("username")
  }
}
```

```

    }
    get Password() {
        return this.loginForm.get("password")
    }

    // Lógica de alternância: decide se deve chamar o método
    de login ou de cadastro
    formAction = () => {
        if(this.isSubscribe)
            return this.subscribe();
        return this.login()
    }

    // --- COMPETÊNCIA 8: REQUISIÇÃO HTTP (LOGIN) ---
    login = () => {
        // Validação de segurança: se o formulário não for váli
do, para o processo aqui
        if(!this.loginForm.valid)
        {
            alert("Nem todos os campos são válidos!");
            return
        }

        // Monta o objeto de dados extraindo os valores atuais
do formulário
        const data: LoginDto = {
            password: this.Password?.value,
            username: this.Username?.value
        }

        // Chama o serviço de API. O .subscribe() é onde a ação
acontece (Competência 7)
        this.api.login(data).subscribe(
            res => {
                // Sucesso: armazena o token de segurança no navega
dor (Competência 10 - Auth)
                sessionStorage.setItem("token", res);
            }
        )
    }

```



```

        // Competência 4: Navega para a página principal (SPA) sem recarregar o browser
        this.router.navigate(['']);
    }
});
}

// Método para cadastrar novo usuário (ainda em desenvolvimento no seu código)
subscribe = () => {
    if(!this.loginForm.valid)
    {
        alert("Nem todos os campos são validos!");
        return
    }
    // Aqui entraria a chamada para this.api.subscribe(data)...
}
}

```

```

// html login page
@if (isSubscribe) {
    <h1>Subscribe</h1>
}@else {
    <h1>Login</h1>
}
<form action="get" [formGroup]="loginForm" (ngSubmit)="formAction()">
    <label for="">Username</label>
    <br>
    <input type="text" formControlName="username">
    <br>
    <label for="">Password</label>
    <br>
    <input type="password" formControlName="password">
    <br>
    <button type="submit">
        @if (isSubscribe) {

```

```

        Subscribe
      }@else {
        Login
      }
    </button>
    <button type="button" (click)="this.isSubscribe = !this.isSubscribe">
      @if (isSubscribe) {
        login
      }@else {
        Subscribe
      }
    </button>
  </form>

```

Signal

O Angular cria uma dependência direta. Se apenas o pixel na posição `[5][10]` mudou através do `.update()`, o framework consegue ser muito mais eficiente na renderização, mantendo a fluidez da tela mesmo com milhares de atualizações por segundo vindas do WebSocket.

```

import { Component, signal } from '@angular/core';
import { RoomApi } from '../../../domain/room.api';
import { ActivatedRoute } from '@angular/router';
import { IPixel } from '../../../main-page/components/pixel/IPixel';
import { Pixel } from '../../../main-page/components/pixel/pixel';
import { Subscription } from 'rxjs';

@Component({
  selector: 'app-especific-room-page',
  standalone: true, // No Angular moderno, componentes costumam ser standalone
  imports: [Pixel], // Importa o componente de visualização do pixel (Competência 2 e 5)

```

```

    templateUrl: './especific-room-page.html',
    styleUrls: ['./especific-room-page.css'],
  })
  export class EspecificRoomPage {

    constructor (
      private api: RoomApi,          // Injeta o serviço de comun
      icação (WebSocket/HTTP)
      private router: ActivatedRoute // Injeta ferramenta par
      a ler o ID da URL (Competência 4 - SPA)
    ){}

    // Atributo para guardar o ID da sala atual
    protected id: string = '';

    // Competência 6: Gerenciamento de estado reativo com Sig
    nal
    // IPixel[][] indica uma matriz (linhas e colunas) para o
    quadro do rPlace
    protected pixels = signal<IPixel[][]>([]);

    // Competência 7: Objeto que permite cancelar a "escuta"
    do Observable depois
    protected pixelSubscription!: Subscription

    ngOnInit(){
      // --- PASSO 1: INICIALIZAÇÃO DO QUADRO (MOCK) ---
      // Cria uma grade 100x100 de pixels cinzas para a tela
      não começar em branco
      let lines = [];
      for (let y = 0; y < 100; y++) {
        let row : IPixel[] = [];
        for(let x = 0; x < 100; x++ ) {
          row.push({
            Color: '#585858',
            X: x,
            Y: y
          })
        }
      }
    }
  }

```

```

    }
    lines.push(row);
  }
  // .set() substitui todo o valor do Signal pelo novo qu
  adro inicial
  this.pixels.set(lines);

  // --- PASSO 2: IDENTIFICAÇÃO DA ROTA ---
  // Pega o ID que está na URL (ex: /room/123) para saber
  qual sala carregar
  this.router.paramMap.forEach((param) => {
    this.id = param.get('id') ?? ""
  })

  // --- PASSO 3: TEMPO REAL (Competência 9 e 10) ---
  // Abre a conexão WebSocket através do serviço para a s
  ala específica
  this.api.connect(this.id)

  // Se inscreve no Observable 'pixels' do serviço para o
  uvir mensagens do servidor
  this.pixelSubscription = this.api.pixels.subscribe(res
=> {
    console.log("subscription updating: ", res);

    // Mensageria: dependendo do 'type' da mensagem, o fr
    ont-end age de forma diferente
    switch (res.type) {
      case 'FULL_LOAD': // Servidor mandou o quadro intei
        ro (geralmente ao entrar)
        this.drawAllApiPixels(res.payload)
        break;
      case 'SINGLE_LOAD': // Alguém pintou apenas um pixe
        l, vamos atualizar só ele
        this.drawSinglePixel(res.payload)
        break;
      default:
        break;
    }
  })

```

```

    }
  })
}

// Lógica para atualizar apenas UM pixel na matriz usando
Signal
drawSinglePixel = (data: IPixel) => {
  console.log("Drawing a single pixel");

  // .update() permite modificar o valor do Signal basean
do-se no valor anterior
  this.pixels.update(oldValues => {
    // Mapeia as linhas e colunas para encontrar a coorde
nada exata (X, Y)
    return oldValues.map((currentRow, y) => {
      return currentRow.map((pixel, x) => {
        if(x == data.X && y == data.Y) {
          return data // Retorna o novo pixel pintado
        } else {
          return pixel // Mantém o pixel antigo
        }
      })
    })
  })
}

// Lógica para injetar uma lista de vários pixels de uma
vez (otimização)
drawAllApiPixels = (data: IPixel[]) => {
  console.log("Drawing all pixels");
  this.pixels.update(oldValues => {
    // Cria uma cópia da matriz atual para não mutar o es
tado diretamente
    var values = [...oldValues];

    // Percorre a lista recebida da API e substitui na po
sição correta
    data.forEach(newPixel => {

```

```

        if (values[newPixel.Y]) {
            values[newPixel.Y][newPixel.X] = newPixel;
        }
    })

    return values; // Retorna a matriz atualizada para o
Signal
    })
}

// Competência 8 e 10: Envia uma requisição de pintura pa
ra o backend
updateData = (data: IPixel) => {
    this.api.updatePixel({
        Pixel: data,
        // Pega o token de autenticação salvo no navegador (A
uth)
        UserToken: sessionStorage.getItem('token') ?? ""
    })
}

// Competência 7 (Controle de Observables): Limpeza
ngOnDestroy(): void {
    // Fecha a conexão do socket para não gastar internet/m
emória à toa
    this.api.closeConnection();
    // Cancela a inscrição (unsubscribe) para evitar Memory
Leaks
    this.pixelSubscription.unsubscribe();
}
}

```

- **Gerenciamento de Memória (`ngOnDestroy`)**: Note que ao final do código há o `unsubscribe()`. Isso é essencial para provar que você sabe controlar a atividade de um Observable (Competência 7). Sem isso, o app continuaria tentando processar pixels mesmo se você saísse da página.

- **Imutabilidade com Signals:** No método `drawSinglePixel`, não alteramos `this.pixels` diretamente com um sinal de igual. Usamos `.update()`. Isso é o que permite ao Angular saber exatamente quando e onde renderizar novamente (Competência 6).
- **Mensageria (Switch Case):** O uso do `switch(res.type)` é o padrão de **Mensageria/Event-Driven Architecture**. O frontend não adivinha o que o dado é; o servidor envia um "tipo" (FULL_LOAD ou SINGLE_LOAD) e o frontend reage conforme o contrato (Competência 10).
- **Matriz Dinâmica:** O uso de `IPixel[][]` (matriz) no Signal é uma forma performática de lidar com o rPlace, pois facilita encontrar a coordenada `[y]` `[x]` rapidamente

```
this.pixels.update(oldValues => { ... })
```

A função do Signal aqui é garantir que você não "atropele" o estado anterior. O método `.update()` recebe o valor atual (`oldValues`), permite que você processe a mudança e retorne um **novo** valor. Isso evita bugs de sincronia onde dois pixels tentam ser pintados ao mesmo tempo.

Como o seu Signal `pixels` armazena uma matriz (`IPixel[][]`), você precisa de dois loops (`@for`) para desenhar o quadro.

`especific-room-page.html`

```
<p>especific-room-page works!</p>
{{id}}
<main class="pixels-zone">
  @for (row of pixels(); track $index) {
    <div class="row">
      @for (pixel of row; track $index) {
        <app-pixel [data]="pixel" (onChange)="updateData($event)" />
      }
    </div>
  }
</main>
```

HTTP Client

```
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class Api {
  // protected readonly URL: string = "http://10.234.197.18:5294/api"
  protected readonly URL: string = "http://localhost:5294/api"

  constructor( protected client: HttpClient ){}

  protected headers: HttpHeaders = new HttpHeaders({
    "Authorization": sessionStorage.getItem('token')!
  })
}
```

Observable

Observable em login

```
import { Injectable } from '@angular/core';
import { Api } from './api';
import { LoginDto } from './UserInterfaces';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root',
})
export class AuthApi extends Api {
  // O retorno do endpoint é o token ou seja uma *string*
  login = (data: LoginDto) : Observable<string> => {
```



```

        return this.client.post<string>(`${this.URL}/auth/login`
, data).pipe();
    }
    subscribe = (data: LoginDto): Observable<void> => {
        return this.client.post<void>(`${this.URL}/auth/subscri
be`, data).pipe()
    }
}

```

Observable criando pixels

```

import { Injectable } from '@angular/core';
import { Api } from './api';
import { Observable, Subject } from 'rxjs';
import { IPixel } from '../features/main-page/components/pixel/IPixel';

@Injectable({
    providedIn: 'root',
})
export class PixelApi extends Api{

    public GetAll = (): Observable<IPixel[]> => {
        return this.client.get<IPixel[]>(`${this.URL}/pixel`).pipe();
    }

    public UpdatePixel = (data: IPixel): Observable<void> =>
    {
        return this.client.post<void>(`${this.URL}/pixel`, data).pipe()
    }

}

```

```

import { Injectable } from '@angular/core';
import { Api } from './api';

```

```

import { Observable, Subject } from 'rxjs';
import { IPixel, IUpdatePixelDto } from '../features/main-page/components/pixel/IPixel';
import { CanvasAction, GetAllRoomsResponse, MessageType, WebSocketMessage } from '../interfaces/room';

@Injectable({
  providedIn: 'root',
})
export class RoomApi extends Api {
  // Endereço do servidor WebSocket. É aqui que o "tubo" de dados em tempo real será aberto.
  private wsURL = "ws://localhost:5294/api/room";

  // Objeto nativo do navegador para lidar com WebSockets (Competência 9)
  private socket!: WebSocket;

  // Competência 7 e 10: O Subject é um tipo especial de Observable que permite
  // "enviar" dados manualmente para quem estiver inscrito.
  private _pixelsSubject = new Subject<CanvasAction>();

  // Expõe o Subject como um Observable para que os componentes não consigam
  // enviar dados por ele, apenas "escutar".
  public pixels: Observable<CanvasAction> = this._pixelsSubject.asObservable();

  // Método para abrir a conexão em tempo real com uma sala específica
  public connect = (roomId: string) => {
    // Autenticação: Pega o token salvo no login para provar quem é o usuário
    const token = sessionStorage.getItem('token');
    if(!token) return

    // Inicializa a conexão WebSocket passando o ID da sala

```

```

e o Token na URL (Protocolo WS)
    this.socket = new WebSocket(`${this.wsURL}/${roomId}?token=${encodeURIComponent(token)}`);

    // Evento disparado quando o "tubo" é aberto com sucesso
    this.socket.onopen = (res) => {
        console.log("Socket Conectado ", res);
    };

    // --- COMPETÊNCIA 10: MENSAGERIA ---
    // Este evento é disparado toda vez que o SERVIDOR envia algo para nós
    this.socket.onmessage = (event: MessageEvent) => {
        // Converte a string JSON recebida de volta para um objeto TypeScript
        const message: WebSocketMessage<any> = JSON.parse(event.data)

        // Analisa o tipo da mensagem para saber o que fazer (Protocolo de Comunicação)
        switch(message.Type) {
            case MessageType.Message: {
                break;
            }
            case MessageType.FirstConnection: {
                // Quando conectamos pela primeira vez, o servidor manda a foto atual do quadro
                // .next() "empurra" o dado para dentro do Observable que o componente está ouvindo
                this._pixelsSubject.next({type: "FULL_LOAD", payload: message.Data})
                break;
            }
            case MessageType.PlayerAction: {
                // Quando algum outro usuário pinta um pixel, recebemos apenas essa mudança

```

```

        this._pixelsSubject.next({type: "SINGLE_LOAD", payload: message.Data})
        break;

    }
    default: {
        console.log("nao caiu em nenhum dos casos");
        break;
    }
}
};

// Tratamento de erros de conexão
this.socket.onerror = (error) => {
    console.log("Erro no websocket", error);
}

// Evento de fechamento: limpa recursos se a conexão cair
this.socket.onclose = (event) => {
    console.log("WebSocket connection closed:", event);
}

// Competência 9: Envia uma ação do usuário (pintar pixel) para o servidor
public updatePixel = (data: IUpdatePixelDto) => {
    // Só tenta enviar se a conexão estiver aberta (OPEN)
    if(this.socket.readyState === WebSocket.OPEN){
        // Converte o objeto de volta para string para trafegar pelo socket
        this.socket.send(JSON.stringify(data));
    }
    else
        console.error("Error on send message to socket" + this.socket.readyState);
}

```

```

    // Finaliza a conexão manualmente (chamado no OnDestroy d
o componente)
    public closeConnection = () => {
        if(this.socket)
            this.socket.close();
    }

    // Competência 8: Requisição HTTP tradicional para buscar
a lista de salas
    public getRooms = () : Observable<GetAllRoomsResponse> =>
{
        var token = sessionStorage.getItem('token')
        if(!token) alert("Unauthorized operation!!")
        // Note o uso do HttpClient (this.client) herdado da cl
asse Api
        return this.client.get<GetAllRoomsResponse>(`${this.UR
L}/room`, {headers:this.headers}).pipe();
    }
}

```

```

import { Component, signal } from '@angular/core';
import { RoomApi } from '../../../domain/room.api';
import { ActivatedRoute } from '@angular/router';
import { IPixel } from '../../../main-page/components/pixel/IP
ixel';
import { Pixel } from "../../../main-page/components/pixel/pix
el";
import { Subscription } from 'rxjs';

@Component({
    selector: 'app-especific-room-page',
    standalone: true, // Componente Standalone (Competência
5)
    imports: [Pixel], // Importa o componente visual do Pixel
para usar no HTML
    templateUrl: './especific-room-page.html',
    styleUrls: ['./especific-room-page.css'],

```

```

}))
export class EspecificRoomPage {

    // O Constructor faz a Injeção de Dependência
    constructor (
        private api: RoomApi,          // Nosso serviço de WebSoc
ket e API (Competência 8 e 9)
        private router: ActivatedRoute // Ferramenta para ler p
arâmetros da URL (Competência 4 - SPA)
    ){}

    // Armazena o ID da sala atual (ex: 'sala-reddit')
    protected id: string = '';

    // --- COMPETÊNCIA 6: SIGNALS ---
    // Criamos um sinal que armazena uma matriz (Array de Arr
ays) de Pixels.
    // Signals são reativos: se mudarmos o valor aqui, o HTML
atualiza automaticamente.
    protected pixels = signal<IPixel[][]>([]);

    // --- COMPETÊNCIA 7: OBSERVABLES ---
    // Variável para guardar a "assinatura" do banco de dados
em tempo real.
    // Precisamos dela para poder cancelar a escuta quando o
usuário sair da página.
    protected pixelSubscription!: Subscription

    ngOnInit(){
        // --- INICIALIZAÇÃO DO QUADRO (ESTADO INICIAL LOCAL) -
--
        // Cria um tabuleiro 100x100 vazio (cinza) para o usuár
io não ver a tela branca.
        let lines = [];
        for (let y = 0; y < 100; y++) {
            let row : IPixel[] = [];
            for(let x = 0; x < 100; x++ ) {
                row.push({

```

```

        Color: '#585858',
        X: x,
        Y: y
    })
}
lines.push(row);
}
// Atualiza o Signal com o quadro cinza inicial.
this.pixels.set(lines);

// --- COMPETÊNCIA 4: ROTEAMENTO (SPA) ---
// Escuta a URL. Se o link for /room/1, ele pega o "1"
e coloca na variável this.id
this.router.paramMap.forEach((param) => {
    this.id = param.get('id') ?? ""
})

// --- COMPETÊNCIA 9: WEBSOCKET ---
// Chama o método no serviço para abrir o "tubo" de com
unicação com o servidor.
this.api.connect(this.id)

// --- COMPETÊNCIA 10: MENSAGERIA ---
// Se inscreve no Observable que vem do serviço.
// Sempre que o servidor enviar um pixel novo, este cód
igo abaixo roda.
this.pixelSubscription = this.api.pixels.subscribe(res
=> {
    console.log("subscription updating: ", res);

    // O Switch decide o que fazer baseado no "tipo" da
mensagem (Contrato de Mensageria)
    switch (res.type) {
        case 'FULL_LOAD':
            // Recebeu o quadro inteiro do servidor (aconte
ce ao entrar na sala)
            this.drawAllApiPixels(res.payload)
            break;

```

```

        case 'SINGLE_LOAD':
            // Recebeu apenas um pixel (alguém pintou agora!)
            this.drawSinglePixel(res.payload)
            break;
        default:
            break;
    }
  })
}

// MÉTODO PARA ATUALIZAR APENAS UM PIXEL (Performance)
drawSinglePixel = (data: IPixel) => {
  console.log("Drawing a single pixel");

  // Usamos .update() do Signal para modificar o estado de
  // forma imutável
  this.pixels.update(oldValues => {
    // Percorremos a matriz atual e substituímos APENAS o
    // pixel que mudou na coordenada X, Y
    return oldValues.map((currentRow, y) => {
      return currentRow.map((pixel, x) => {
        if(x == data.X && y == data.Y) {
          return data // Retorna o pixel novo enviado pelo
            // servidor
        } else {
          return pixel // Mantém o pixel que já estava lá
        }
      })
    })
  })
}

// MÉTODO PARA CARREGAR O TABULEIRO INTEIRO
drawAllApiPixels = (data: IPixel[]) => {
  console.log("Drawing all pixels");
  this.pixels.update(oldValues => {
    // Cria uma cópia da matriz atual para manipular

```



```

    var values = [...oldValues];

    // Percorre a lista de pixels da API e substitui nas
    posições corretas (Y e X)
    data.forEach(newPixel => {
        values[newPixel.Y][newPixel.X] = newPixel;
    })

    return values; // Devolve a matriz completa e atualiz
    ada para o Signal
    })
    }

    // MÉTODO CHAMADO QUANDO O USUÁRIO CLICA PARA PINTAR (Com
    petência 8 e 9)
    updateData = (data: IPixel) => {
        this.api.updatePixel({
            Pixel: data,
            // Pega o Token do banco de dados local do navegador
            (Segurança/Auth)
            UserToken: sessionStorage.getItem('token') ?? ""
        })
    }

    // --- CONTROLE DE OBSERVABLES (CICLO DE VIDA) ---
    // Este método roda quando o usuário fecha a página ou mu
    da de rota.
    ngOnDestroy(): void {
        // Fecha o WebSocket (para não gastar internet à toa)
        this.api.closeConnection();
        // PARA a atividade do Observable (Essencial para não t
        ravar a memória do PC)
        this.pixelSubscription.unsubscribe();
    }
}

```

Subscription

```
import { Component, OnInit } from '@angular/core';
// Importação dos serviços de roteamento: Router (para navegar) e ActivatedRoute (para ler a URL atual)
import { ActivatedRoute, Router } from '@angular/router';

@Component({
  selector: 'app-login-page',
  templateUrl: './login-page.component.html',
  styleUrls: ['./login-page.component.css']
})
// Implementa OnInit para rodar lógica assim que o componente for criado
export class LoginPageComponent implements OnInit {

  // Variável de controle (flag): define se o usuário quer "entrar" ou "se cadastrar"
  isSubscribe = false;

  // Uma variável simples inicializada com uma string (provavelmente para fins de teste ou exibição)
  login: string = "Nycollas";

  constructor(
    // Injeção de dependência: o "_" é uma convenção para indicar que são serviços privados
    private _router: Router,
    private _activatedRoute: ActivatedRoute
  ) {}

  // --- COMPETÊNCIA 4 e 7: SPA e OBSERVABLES ---
  ngOnInit(): void {
    // Escutamos os parâmetros da rota (URL).
    // .params é um Observable: ele "reage" toda vez que a URL muda sem recarregar a página.
    this._activatedRoute.params.subscribe((param) => {
```

```

        // Lógica de Negócio: Se na URL houver um parâmetro c
hamado 'tab' igual a 'subscribe',
        // a variável 'isSubscribe' vira true. Caso contrári
o, false.
        // Exemplo de URL: /login/subscribe -> isSubscribe =
true
        this.isSubscribe = param['tab'] == "subscribe";

    })
}
}

```

Pipes

```

import { Injectable } from '@angular/core';
import { Api } from './api';
import { LoginDto } from './UserInterfaces';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root',
})
export class AuthApi extends Api {
  // O retorno do endpoint é o token ou seja uma *string*
  login = (data: LoginDto) : Observable<string> => {
    return this.client.post<string>(`${this.URL}/auth/login
`, data).pipe();
  }
  subscribe = (data: LoginDto): Observable<void> => {
    return this.client.post<void>(`${this.URL}/auth/subscri
be`, data).pipe()
  }
}

```

```

import { Injectable } from '@angular/core';
import { Api } from './api';

```

```

import { Observable, Subject } from 'rxjs';
import { IPixel } from '../features/main-page/components/pixel/IPixel';

@Injectable({
  providedIn: 'root',
})
export class PixelApi extends Api{

  public GetAll = (): Observable<IPixel[]> => {
    return this.client.get<IPixel[]>(`${this.URL}/pixel`).pipe();
  }

  public UpdatePixel = (data: IPixel): Observable<void> =>
  {
    return this.client.post<void>(`${this.URL}/pixel`, data).pipe()
  }

}

```

WebSocket

```

import { Injectable } from '@angular/core';
import { Api } from './api'; // Classe pai que contém a URL base e o HttpClient
import { Observable, Subject } from 'rxjs'; // Ferramentas para programação reativa (Competência 7)
import { IPixel, IUpdatePixelDto } from '../features/main-page/components/pixel/IPixel';
import { CanvasAction, GetAllRoomsResponse, MessageType, WebSocketMessage } from './interfaces/room';

@Injectable({
  providedIn: 'root', // O serviço é um Singleton (instância

```

```

a única) disponível em todo o app
}))
export class RoomApi extends Api {
  // URL do servidor de WebSocket (protocolo ws:// em vez de http://)
  private wsURL = "ws://localhost:5294/api/room";

  // Objeto nativo que gerencia a conexão persistente com o servidor (Competência 9)
  private socket!: WebSocket;

  // --- COMPETÊNCIA 7 e 10: MENSAGERIA ---
  // O Subject é um "produtor" de eventos. Ele permite emitir dados para múltiplos ouvintes.
  // É privado para que apenas este serviço possa enviar dados para o socket.
  private _pixelsSubject = new Subject<CanvasAction>();

  // Expõe o Subject como um Observable (somente leitura) para os componentes.
  // Assim, o componente apenas "consome" os dados que chegam do servidor.
  public pixels: Observable<CanvasAction> = this._pixelsSubject.asObservable();

  // Método para abrir a conexão WebSocket (Competência 9)
  public connect = (roomId: string) => {
    // Autenticação: Recupera o token do armazenamento do navegador
    const token = sessionStorage.getItem('token');
    if(!token)
      return

    // Cria a conexão enviando o Token via Query String para a validação no backend
    this.socket = new WebSocket(`${this.wsURL}/${roomId}?token=${encodeURIComponent(token)}`);
  }
}

```

```

    // Callback: Executa quando a conexão é estabelecida co
m sucesso
    this.socket.onopen = (res) => {
        console.log("Socket Connectado ", res);
    };

    // --- COMPETÊNCIA 10: TRATAMENTO DE MENSAGENS (EVENT-D
RIVEN) ---
    // Executa toda vez que o servidor envia um pacote de d
ados para o cliente
    this.socket.onmessage = (event: MessageEvent) => {
        // Converte o texto JSON recebido em um objeto tipado
        const message: WebSocketMessage<any> = JSON.parse(eve
nt.data)
        console.log(message);

        // Lógica de Mensageria: Age de acordo com o "tipo" d
a mensagem recebida
        switch(message.Type) {
            case MessageType.Message: {
                // Mensagem de texto simples (Chat ou Log)
                break;
            }
            case MessageType.FirstConnection: {
                // Recebe o estado atual completo do canvas (Comp
etência 6)
                // .next() notifica todos os componentes inscrito
s que o quadro chegou
                this._pixelsSubject.next({type: "FULL_LOAD", payl
oad: message.Data})
                break;
            }
            case MessageType.PlayerAction: {
                // Recebe a atualização de um único pixel alterad
o por outro usuário
                this._pixelsSubject.next({type: "SINGLE_LOAD", pa
yload: message.Data})

```

```

        break;

    }
    default: {
        console.log("nao caiu em nenhum dos casos");
        break;
    }
}
};

// Tratamento de erros de rede ou protocolo
this.socket.onerror = (error) => {
    console.log("Erro no websocket", error);
}

// Callback executado quando a conexão cai ou é fechada
propositalmente
this.socket.onclose = (event) => {
    console.log("WebSocket connection closed:", event);
}

// Envia dados do Cliente para o Servidor através do socket (Competência 9)
public updatePixel = (data: IUpdatePixelDto) => {
    // Verifica se a conexão está pronta (Estado 1 - OPEN)
    antes de tentar enviar
    if(this.socket.readyState === WebSocket.OPEN){
        // Transforma o objeto DTO em String JSON para o envio
        this.socket.send(JSON.stringify(data));
    }
    else
        console.error("Error on send message to socket" + this.socket.readyState);
}

// Método para encerrar a conexão (usado no OnDestroy do

```

```

componente)
  public closeConnection = () => {
    if(this.socket)
      this.socket.close();
  }

  // --- COMPETÊNCIA 8: REQUISIÇÃO HTTP (REST) ---
  // Diferente do WebSocket, esta é uma requisição comum de
  ida e volta
  public getRooms = () : Observable<GetAllRoomsResponse> =>
  {
    var token = sessionStorage.getItem('token')
    if(!token) alert("Unauthorized operation!!")

    // Faz um GET simples para buscar a lista de salas disp
    oníveis
    // .pipe() vazio: ponto de extensão para futuros operad
    ores RxJS
    return this.client.get<GetAllRoomsResponse>(`${this.UR
    L}/room`, {headers:this.headers}).pipe();
  }
}

```

```

import { IPixel, UserDto } from "../../features/main-page/c
omponents/pixel/IPixel";

export interface IRoom {
  id: string,
  name: string,
  players: UserDto[],
  pixels: IPixel[]
}

export interface MinimalizedRoom {
  id: string,
  name: string,
  quantityOfPlayers: number,

```



```

}

export interface GetAllRoomsResponse {
  rooms?: MinimalizedRoom[]
}

export interface WebSocketMessage<T> {
  Type: MessageType,
  Data: T
}

export enum MessageType {
  Message,
  PlayerAction,
  FirstConnection
}

export type CanvasAction =
  | {type: "FULL_LOAD"; payload: IPixel[]}
  | {type: "SINGLE_LOAD"; payload: IPixel}

```

```

import { Component, signal } from '@angular/core';
import { RoomApi } from '../../../domain/room.api';
import { ActivatedRoute } from '@angular/router';
import { IPixel } from '../../../main-page/components/pixel/IPixel';
import { Pixel } from '../../../main-page/components/pixel/pixel';
import { Subscription } from 'rxjs';

@Component({
  selector: 'app-especific-room-page',
  imports: [Pixel],
  templateUrl: './especific-room-page.html',
  styleUrls: ['./especific-room-page.css'],
})
export class EspecificRoomPage {

```

```

constructor (
  private api: RoomApi,
  private router: ActivatedRoute
){}

protected id: string = '';
protected pixels = signal<IPixel[][]>([]);

protected pixelSubscription!: Subscription

ngOnInit(){
  //Definindo pixels vazios default por enquanto
  let lines = [];
  for (let y = 0; y < 100; y++) {
    let row : IPixel[] = [];
    for(let x = 0; x < 100; x++ ) {
      row.push({
        Color: '#585858',
        X: x,
        Y: y
      })
    }
    lines.push(row);
  }
  this.pixels.set(lines);

  // pegando id da room para verificar os pixels
  this.router.paramMap.forEach((param) => {
    this.id = param.get('id') ?? ""
  })

  //Conectando com websocket
  this.api.connect(this.id)
  //! UTILIZANDO O OBSERVABLE criado dos pixels
  this.pixelSubscription = this.api.pixels.subscribe(res
=> {
    console.log("subscription updating: ",res);
  }

```

```

        switch (res.type) {
            case 'FULL_LOAD':
                this.drawAllApiPixels(res.payload)
                break;
            case 'SINGLE_LOAD':
                this.drawSinglePixel(res.payload)
                break;

            default:
                break;
        }
    })
}

drawSinglePixel = (data: IPixel) => {
    console.log("Drawing a single pixel");

    this.pixels.update(oldValues => {
        return oldValues.map((currentRow, y) => {
            return currentRow.map((pixel, x) => {
                if(x == data.X && y == data.Y) {
                    return data
                }else {
                    return pixel
                }
            })
        })
    })
}

drawAllApiPixels = (data: IPixel[]) => {
    console.log("Drawing all pixels");
    this.pixels.update(oldValues => {
        var values = [...oldValues];

        data.forEach(newPixel => {
            values[newPixel.Y][newPixel.X] = newPixel;

```

```

    })

    return values;
  })
}

updateData = (data: IPixel) => {
  this.api.updatePixel({
    Pixel: data,
    UserToken: sessionStorage.getItem('token') ?? ""
  })
}

ngOnDestroy(): void {
  this.api.closeConnection();
  this.pixelSubscription.unsubscribe();
}
}

```

Guard

```

import { inject } from '@angular/core';
import { CanMatchFn, Router } from '@angular/router';

export const authGuard: CanMatchFn = (route, segments) => {
  const router = inject( Router )

  const token = sessionStorage.getItem('token') ?? "";
  const logged = "" !== token;

  if(route.path === "login") {
    if(logged){
      return router.createUrlTree([""]);
    }else {
      return true;
    }
  }
}

```

```
}

if(logged) {
    return true;
}
return router.createUrlTree(["login"]);
};
```